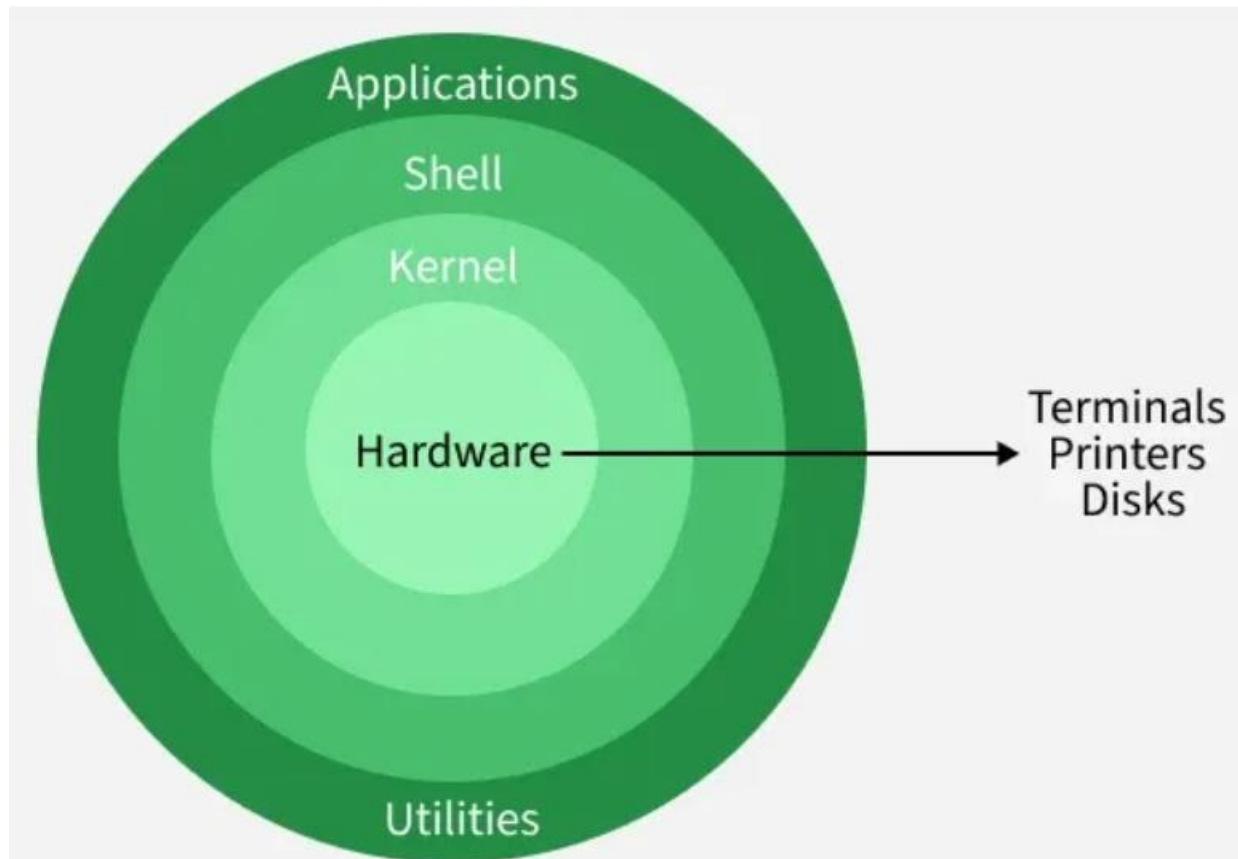


Unit-2(Basic of linux and window system administration)(6 hours)

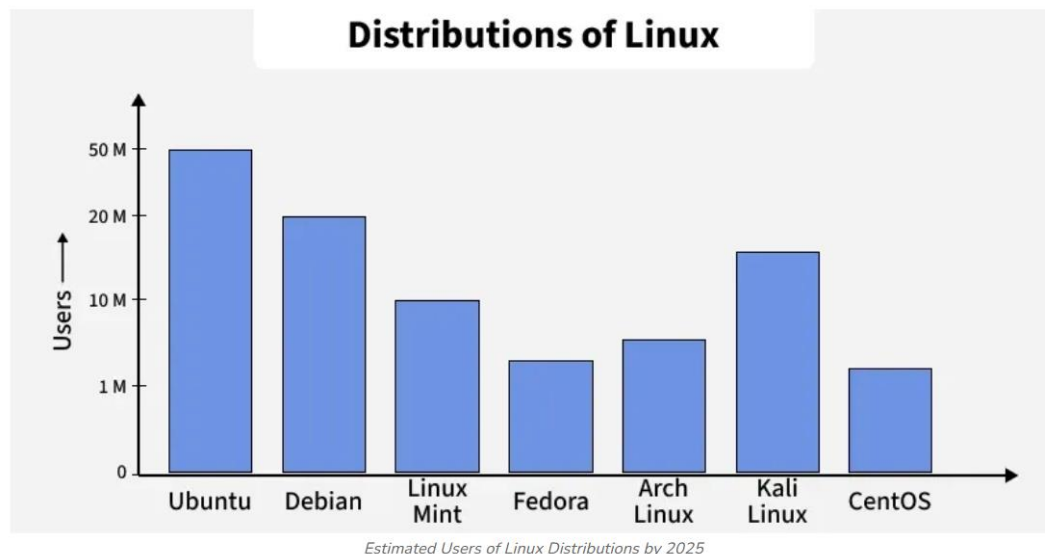
Linux Architecture



A Linux distribution is an operating system built on the Linux kernel, bundled with essential software, tools, and package managers, customized to serve different users like developers, enterprises, cybersecurity professionals, and general users.

- Includes the Linux kernel, system tools, package manager, and optional desktop environment.
- Comes with pre-installed tools, reducing manual setup.
- Built for different purposes like hacking, servers, development, or desktop use.
- Offers flexibility and customization based on user needs.

Top Linux Distributions



Benefits of Using Linux Distributions

1. Cost-Free and Open Source

- Most Linux distributions are free to use and don't require licensing fees.
- They are open-source, allowing users to view, modify, and distribute the code.

2. Customizable

- Linux allows users to modify the operating system's appearance and functionality according to their needs.
- Users can choose from a variety of desktop environments, file systems, and tools.

3. Security

- Linux is highly secure and less vulnerable to viruses and malware compared to many other operating systems.
- Regular updates and strong user permissions enhance system security.

4. Resource-Efficient

- Linux can run efficiently on older or low-spec hardware, extending the usability of older machines.

- Lightweight distributions like Ubuntu and Puppy Linux are designed for systems with limited resources.

5. Development and Programming

- Linux distributions are ideal for software development due to their compatibility with programming languages and tools.
- Many distributions include pre-installed compilers, editors, and debugging tools.

6. Stability and Reliability

- Linux distributions are known for their stability, especially in server environments.
- They are widely used in mission-critical systems due to their consistent performance.

Ubuntu vs CentOS

Feature	☐ Ubuntu	● CentOS
Base	Debian-based	Red Hat Enterprise Linux (RHEL)-based
Developer	Canonical Ltd.	Community (now replaced by CentOS Stream)
Ease of Use	Very user-friendly	More complex, server-focused
Interface	Good GUI + CLI	Mostly CLI (server use)
Package Manager	apt	yum / dnf
Stability	Stable but frequently updated	Very stable, long-term support
Updates	Regular updates	Slower, enterprise-style updates
Usage	Desktop + Server	Mainly Server & enterprise systems
Learning	Good for beginners	Better for advanced/server admins

Basic Linux Commands

Linux commands are used to interact with the operating system through the terminal and perform tasks like file management, navigation, and system monitoring. Learning basic Linux commands helps beginners understand how Linux works and use it efficiently for daily tasks.

- Helps beginners understand and use the Linux terminal effectively
- Covers commonly used commands for files, directories, and system tasks
- Useful for students, developers, and system administrators
- Builds a strong foundation for advanced Linux and server management

25 Basic Linux Commands				
ls	cp	touch	man	sort
pwd	mv	ln	grep	cal
mkdir	rm	cat	echo	whereis
cd	uname	clear	wget	df
rmdir	locate	ps	whoami	wc

25 Basic Linux Commands

1. ls

The `ls` command in Linux is used to list files and directories present in the current working directory.

- Helps users quickly understand directory contents.

Syntax:

```
ls [options] [directory]
```

Example:

```
ls
```

Output:

```
sahil@sahil:~/GFG$ ls
file1.txt  file2.txt  folder1  folder2
sahil@sahil:~/GFG$
```

Output

- The `ls` command lists all the directories and files available in the current working directory.

2. pwd

The [pwd](#) command in Linux is used to display the full path of the current directory.

Syntax:

```
pwd
```

Example:

```
pwd
```

Output:

```
sahil@sahil:~/GFG$ pwd
/home/sahil/GFG
sahil@sahil:~/GFG$
```

Output

- The pwd command prints the absolute path of the current working directory.

3. mkdir

The [mkdir](#) command in Linux is used to create new directories or folders.

Syntax:

```
mkdir directory_name
```

Example:

```
mkdir GeeksforGeeks
```

Output:

```
sahil@sahil:~/GFG$ mkdir GeeksforGeeks
sahil@sahil:~/GFG$ ls
file1.txt  file2.txt  folder1  folder2  GeeksforGeeks
sahil@sahil:~/GFG$
```

Output

- The mkdir command creates a new directory named GeeksforGeeks.

4. cd

The [cd](#) command in Linux is used to change the current working directory.

- Allows navigation between directories.
- Moves the user to the home directory when used without arguments.

Syntax:

```
cd directory_name
```

Example:

```
cd GeeksforGeeks
```

Output:

```
sahil@sahil:~/GFG$ cd GeeksforGeeks/  
sahil@sahil:~/GFG/GeeksforGeeks$
```

Output

- The cd command changes the current working directory to GeeksforGeeks.

5. rmdir

The [rmdir](#) command in Linux is used to delete empty directories.

Syntax:

```
rmdir directory_name
```

Example:

```
rmdir TestFolder
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ ls  
testFile.txt  TestFolder  
sahil@sahil:~/GFG/GeeksforGeeks$ rmdir TestFolder  
sahil@sahil:~/GFG/GeeksforGeeks$ ls  
testFile.txt
```

Output

- The rmdir command deletes the empty directory TestFolder.

6. cat

The `cat` command in Linux is used to display file contents and combine multiple files.

Syntax:

```
cat file_name
```

Example:

```
cat test.txt
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ cat test.txt  
Hello Geeks
```

Output

- The cat command displays the content of the file.

7. cp

The `cp` command in Linux is used to copy files or directories.

Syntax:

```
cp source destination
```

Example:

```
cp file1.txt file2.txt
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ cat file1.txt  
Hello from file1  
sahil@sahil:~/GFG/GeeksforGeeks$ cat file2.txt  
sahil@sahil:~/GFG/GeeksforGeeks$ cp file1.txt file2.txt  
sahil@sahil:~/GFG/GeeksforGeeks$ cat file2.txt  
Hello from file1
```

Output

- The cp command copies the contents of file1.txt into file2.txt.

8. mv

The `mv` command in Linux is used to move or rename files and directories.

Syntax:

```
mv old_name new_name
```

Example:

```
mv old.txt new.txt
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ ls
old.txt
sahil@sahil:~/GFG/GeeksforGeeks$ mv old.txt new.txt
sahil@sahil:~/GFG/GeeksforGeeks$ ls
new.txt
sahil@sahil:~/GFG/GeeksforGeeks$
```

Output

- The mv command renames the file old.txt to new.txt.

9. rm

The `rm` command in Linux is used to delete files permanently.

Syntax:

```
rm file_name
```

Example:

```
rm demo.txt
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ ls
demo.txt  new.txt
sahil@sahil:~/GFG/GeeksforGeeks$ rm demo.txt
sahil@sahil:~/GFG/GeeksforGeeks$ ls
new.txt
sahil@sahil:~/GFG/GeeksforGeeks$
```

Output

- The `rm` command deletes the file `demo.txt`.

10. uname

The [uname](#) command in Linux is used to display system information.

- Shows operating system details.
- Helps identify the system.

Syntax:

```
uname [OPTIONS]
```

Example:

```
uname
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ uname  
Linux
```

Output

- The uname command displays the operating system name.

11. locate

The [locate](#) command in Linux is used to find files using a database.

Syntax:

```
locate file_name
```

Example:

```
locate demo.txt
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ locate demo.txt  
/home/sahil/GFG/GeeksforGeeks/demo.txt
```

Output

- The locate command shows the path of the searched file.
- Requires updatedb database

12. touch

The [touch](#) command in Linux is used to create empty files and updates file timestamps.

Syntax:

```
touch file_name
```

Example:

```
touch test.txt
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ ls
sahil@sahil:~/GFG/GeeksforGeeks$ touch test.txt
sahil@sahil:~/GFG/GeeksforGeeks$ ls
test.txt
```

Output

- The touch command creates an empty file named test.txt.

13. ln

The [ln](#) command in Linux is used to create links between files.

- Supports hard and soft [links](#).

Syntax:

```
ln -s source link_name
```

Example:

```
ln -s file1.txt link1.txt
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ ln -s file1.txt link1.txt
sahil@sahil:~/GFG/GeeksforGeeks$ ls
file1.txt link1.txt test.txt
```

Output

- The ln command creates a symbolic link.

14. clear

The [clear](#) command in Linux is used to clear the terminal screen.

Syntax:

```
clear
```

Example:

```
clear
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ ls  
demo.txt  file1.txt  link1.txt  test.txt  
sahil@sahil:~/GFG/GeeksforGeeks$ clear
```

Output

- The terminal screen is cleared.

15. ps

The ps command in Linux is used to display running processes.

- Helps monitor system activity.

Syntax:

```
ps
```

Example:

```
ps
```

Output:

```
sahil@sahil:~/GFG/GeeksforGeeks$ ps
  PID TTY          TIME CMD
 2774 pts/0        00:00:00 bash
 6382 pts/0        00:00:00 ps
```

Output

- The ps command displays currently running processes.
- commonly used as "ps aux"

User Management in Linux

User management in Linux refers to the process of creating, modifying, deleting, and managing user accounts. It allows system administrators to control who can access the system and what resources they can use.

Why User Management is Important

- Provides secure access to the system.
- Controls permissions and ownership of files.
- Supports multiple users on the same system.
- Enhances system security and accountability.

User Account Information

Linux stores user account information in:

/etc/passwd

Example entry:

```
john:x:1001:1001:John Doe:/home/john:/bin/bash
```

Fields:

Field	Description
john	Username
x	Password placeholder
1001	User ID (UID)
1001	Group ID (GID)
John Doe	User description
/home/john	Home directory
/bin/bash	Login shell

Types of Users in Linux

1. Root User

- Superuser with full privileges.
- UID = 0
- Can perform all administrative tasks.

Example:

```
sudo su
```

2. Regular User

- Created for normal system use.
- Limited permissions.

Example:

```
john  
alice  
student
```

3. System User

- Used by services and applications.
- Usually have low UID values.

Examples:

```
www-data  
mysql  
daemon
```

Creating a User

Command:

```
sudo useradd username
```

Example:


```
sudo useradd john
```

Create a home directory:

```
sudo useradd -m john
```

Explanation:

- useradd → Creates a new user.
- -m → Creates a home directory.

Setting a Password

Command:

```
sudo passwd john
```

Example Output:

New password:

Retype new password:

passwd: password updated successfully

Viewing User Information

Using id Command

```
id john
```

Output:

```
uid=1001(john) gid=1001(john) groups=1001(john)
```

Display Current User

```
whoami
```

Output:

```
john
```

Modifying a User

Change Username

sudo usermod -l newname oldname

Example:

sudo usermod -l jack john

Change Home Directory

sudo usermod -d /home/jack -m jack

- -d → New home directory.
- -m → Move existing files.

Change Login Shell

sudo usermod -s /bin/bash jack

Locking and Unlocking Accounts

Lock User

sudo passwd -l john

Prevents login.

Unlock User

sudo passwd -u john

Allows login again.

Deleting a User

Delete user account:

sudo userdel john

Delete user and home directory:

sudo userdel -r john

- -r removes the user's files and home directory.

Listing Users

Display all users:

```
cat /etc/passwd
```

Switching Users

Switch to another user:

```
su username
```

Example:

```
su john
```

Switch to root:

```
sudo su
```

User Password Aging

View password settings:

```
sudo chage -l john
```

Set password expiry:

```
sudo chage -M 90 john
```

Meaning:

- Password expires after 90 days.

Group Management in Linux

A **group** in Linux is a collection of users. Groups are used to manage permissions and access control for files, directories, and system resources. Instead of assigning permissions to each user individually, permissions can be assigned to a group, making administration easier.

Why Group Management is Important

- Simplifies permission management.
- Allows multiple users to share resources.
- Improves security and organization.
- Reduces administrative workload.

Types of Groups in Linux

1. Primary Group

- Every user belongs to one primary group.
- Assigned when the user account is created.
- Files created by the user are associated with this group by default.

Example:

```
id john
```

Output:

```
uid=1001(john) gid=1001(john) groups=1001(john)
```

Here, gid=1001(john) is the primary group.

2. Secondary (Supplementary) Groups

- Additional groups a user can belong to.
- Used to grant extra permissions.

Example:

```
uid=1001(john) gid=1001(john) groups=1001(john),27(sudo)
```

Here:

- john = Primary group
- sudo = Secondary group

Group Information File

Linux stores group information in:

/etc/group

Example entry:

developers:x:1002:john,mary

Fields:

Field	Description
developers	Group name
x	Password placeholder
1002	Group ID (GID)
john,mary	Group members

Creating a Group

Command:

sudo groupadd developers

Explanation:

- groupadd → Creates a new group.
- developers → Group name.

Verify:

grep developers /etc/group

Output:

developers:x:1002:

Viewing Groups

Display Current User's Groups

groups

Example Output:

john sudo developers

Display Groups of a Specific User

groups john

View All Groups

cat /etc/group

Adding a User to a Group

Command:

sudo usermod -aG developers john

Explanation:

- usermod → Modify user.
- -a → Append.
- -G → Secondary groups.
- developers → Group name.
- john → Username.

Verify:

groups john

Output:

john : john developers

Adding a User to Multiple Groups

sudo usermod -aG developers,sudo,docker john

Now John belongs to three additional groups.

Removing a User from a Group

Command:

```
sudo gpasswd -d john developers
```

Output:

Removing user john from group developers

Verify:

```
groups john
```

Changing a User's Primary Group

Command:

```
sudo usermod -g developers john
```

Explanation:

- -g changes the primary group.

Verify:

```
id john
```

Output:

```
uid=1001(john) gid=1002(developers)
```

Renaming a Group

Command:

```
sudo groupmod -n programmers developers
```

Explanation:

- groupmod → Modify group.
- -n → New group name.

Deleting a Group

Command:

```
sudo groupdel developers
```

Verify:

```
grep developers /etc/group
```

No output means the group was deleted.

Group Password Management

Set a group password:

```
sudo gpasswd developers
```

Remove group password:

```
sudo gpasswd -r developers
```

Group Ownership of Files

Change file group ownership:

```
sudo chgrp developers project.txt
```

Verify:

```
ls -l project.txt
```

Output:

```
-rw-r--r-- 1 john developers 500 Jun 25 project.txt
```


Linux File System

- The Linux File System is a structured method of storing and organizing data on a Linux machine. It arranges files in a hierarchical directory format starting from the root directory /.
 - All files and directories in Linux originate from a single root directory /.
 - It follows a [hierarchical](#) tree structure, making navigation simple and logical.
 - Different directories like /home, /etc, /bin, and /var serve specific system purposes.
 - Linux supports multiple file system types like ext4, XFS, Btrfs, each offering different features.

Linux File System Structure

The Linux file system architecture consists of **three layers** that work together to manage file operations.

1. Logical File System (LFS)

- Interface between **user applications** and the file system.
- Handles file operations such as **open, read, write, and close**.
- Performs **permission checks** and **access control**.

Example: When an application opens a file, the Logical File System verifies the user's permissions.

2. Virtual File System (VFS)

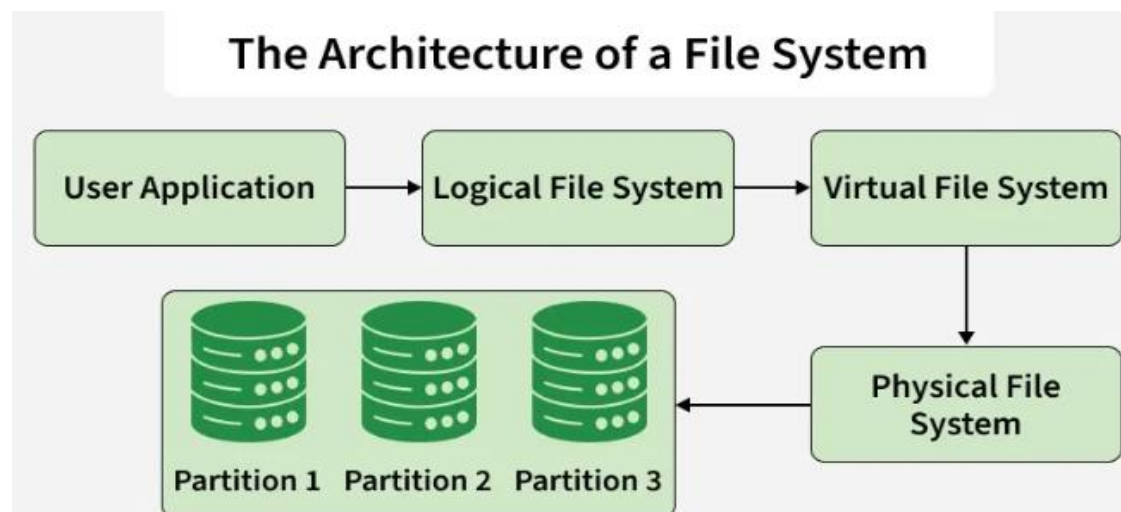
- Provides a **common interface** for different file systems.
- Supports multiple file systems such as **ext4, XFS, Btrfs, FAT32, and NTFS**.
- Hides the implementation details of individual file systems from applications.

Example: An application can access files on ext4 or NTFS using the same system calls.

3. Physical File System

- Interacts directly with the **storage hardware (HDD/SSD)**.
- Manages **data blocks, and disk space allocation**.
- Performs the actual **reading and writing of data**.

Example: Stores `report.txt` on the hard disk and retrieves it when requested.



Linux File Hierarchy Structure

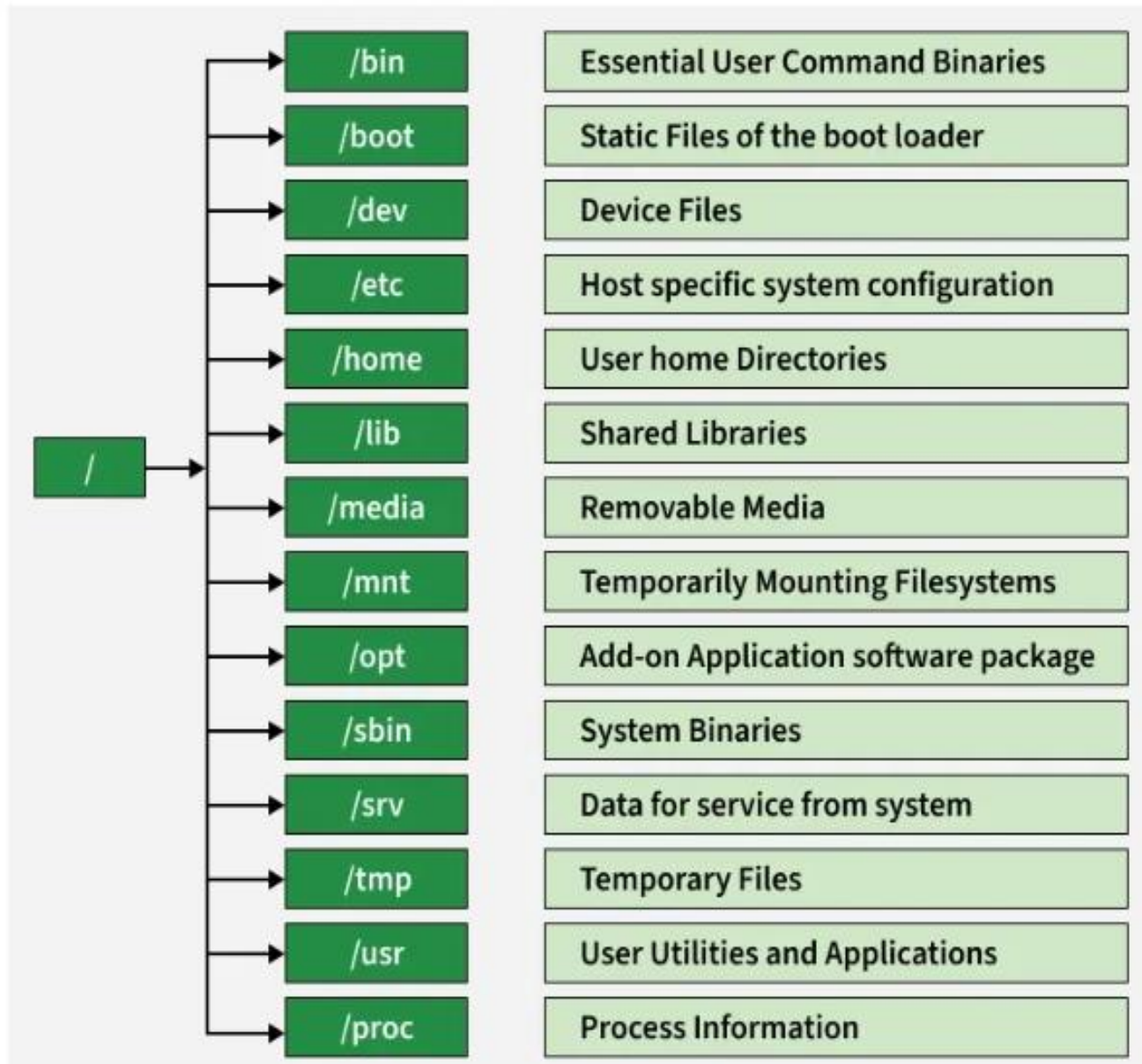
The **Linux File Hierarchy Structure (FHS)** is a **standard** that defines how files and directories should be organized in a Linux operating system.

It ensures that Linux distributions (such as Ubuntu, CentOS, and Debian) use a **consistent directory structure**, making system administration easier.

- **FHS (Filesystem Hierarchy Standard)** defines the standard directory structure in Linux.
- It is maintained by the **Linux Foundation**.
- **All files and directories start from the root directory (/).**
- Files may be stored on different physical or virtual storage devices, but they all appear under the single root directory (/).

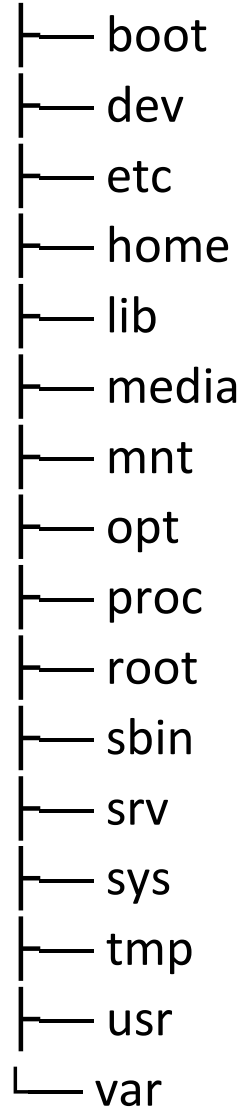
.

•



/

|— bin



Important Directories

Directory	Purpose
/	Root directory; top of the filesystem hierarchy.
/bin	Essential user commands (e.g., <code>ls</code> , <code>cp</code> , <code>mv</code>).
/sbin	System administration commands (e.g., <code>fdisk</code> , <code>ifconfig</code>).
/boot	Bootloader files, Linux kernel, and startup files.
/dev	Device files representing hardware devices.
/etc	System-wide configuration files.
/home	User home directories (e.g., <code>/home/john</code>).
/root	Home directory of the root (administrator) user.
/lib, /lib64	Essential shared libraries and kernel modules.
/media	Mount points for removable media (USB drives, CDs).
/mnt	Temporary mount points.
/opt	Optional third-party software packages.
/proc	Virtual filesystem containing process and kernel information.

Directory	Purpose
<code>/srv</code>	Data served by services (web, FTP, etc.).
<code>/sys</code>	Virtual filesystem providing kernel and hardware information.
<code>/tmp</code>	Temporary files; often cleared on reboot.
<code>/usr</code>	User applications, libraries, and documentation.
<code>/var</code>	Variable data such as logs, mail, databases, and spool files.

1. / (Root Directory)

The **root directory** is the topmost directory in Linux. Every other file and directory is stored inside it.

Example:

```
/
├── home
├── etc
├── usr
└── var
```

Purpose

- Starting point of the entire Linux filesystem.
- All files and folders are located somewhere under `/`.

2. /bin (Binary Commands)

bin stands for **Binary**.

It stores **essential commands** that every user needs.

Examples:

```
ls
cp
mv
cat
pwd
echo
mkdir
rm
```

Example:

```
ls
```

Lists files.

```
cp file1 file2
```

Copies a file.

```
mv file1 folder/
```

Moves or renames a file.

3. /sbin (System Binary)

Contains commands mainly used by the **system administrator (root)**.

Examples:

fdisk
fsck
mkfs
reboot
shutdown

Example:

```
fsck /dev/sda1
```

Checks and repairs a filesystem.

Example:

```
shutdown now
```

Turns off the computer.

Difference

/bin → Commands for everyone

/sbin → Commands mainly for system
administration

4. /boot

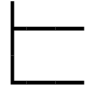
Contains everything needed to **start Linux**.

It stores:

- Linux kernel
- Bootloader files
- Initial RAM disk

Example:

```
/boot  
├── vmlinuz (The compressed Linux kernel.)
```

 `initrd.img` (A temporary filesystem loaded into RAM during startup.)
`grub` (Contains the **GRUB bootloader** configuration files.)

Similarly,

`/boot`

starts Linux.

Without `/boot`, Linux cannot boot properly.

5. `/dev` (Devices)

Linux treats hardware as files.

This directory contains files representing devices.

Examples:

`/dev/sda`

Hard disk

`/dev/sdb`

Generates random numbers.

Example:

```
cat /dev/random
```

Produces random characters.

Think of `/dev` as a folder containing "virtual files" for all hardware devices.

6. /etc (Configuration Files)

Stores **system configuration files**.

Examples:

`/etc/passwd`

User account information

`/etc/hosts`

Hostname mappings

`/etc/fstab`

Disk mount information

`/etc/ssh`

SSH server configuration

7. /home

Contains personal folders for users.

Example:

```
/home
├── alice
├── john
└── ram
```

Each user stores:

- Documents
- Music
- Downloads

- Pictures
- Desktop files

Example:

```
/home/john/Documents
```

8. /root

Home directory of the **root (administrator)**.

Example:

```
/root
```

Only the administrator normally uses it.

Difference:

```
/home/john
```

Normal user

```
/root
```

Administrator

9. /lib and /lib64

Contain essential libraries needed by programs.

A library is a collection of reusable code.

Example:

When you run:

```
ls
```

Linux loads required libraries from:

/lib

Without these libraries, programs won't run.

10. /media

Used for **removable storage devices**.

Examples:

- USB drive
- CD/DVD
- Memory card

When you insert a USB drive, it may appear as:

/media/user/USB

11. /mnt

Used for **temporary mounts**.

Example:

An administrator manually mounts another hard drive:

```
mount /dev/sdb1 /mnt
```

Difference:

/media

Automatic mounts (USBs)

/mnt

Manual or temporary mounts

12. /opt

Stores **optional or third-party software**.

Examples:

- Google Chrome
- Zoom
- Oracle software
- Custom applications

Example:

```
/opt/google
```

Think of it as an "extra software" folder.

13. /proc

A **virtual filesystem** that shows information about the system and running processes. The files are created in memory while the system is running.

Examples:

```
/proc/cpuinfo
```

CPU information

```
/proc/meminfo
```

Memory information

```
/proc/version
```

Linux version

Command:

```
cat /proc/cpuinfo
```

Shows processor details.

14. `/srv`

Contains data provided by servers and services.

Examples:

- Website files
- FTP files
- File-sharing data

Example:

`/srv/www`

Stores website files.

16. `/sys`

Another **virtual filesystem** that provides information about hardware devices and the Linux kernel. It also allows some hardware settings to be viewed or changed.

Examples:

`/sys/class`

Device classes

`/sys/block`

Disk information

`/sys/devices`

Hardware devices

Think of `/sys` as the hardware control panel of Linux.

17. `/tmp`

Stores **temporary files** created by programs.

Examples:

- Temporary downloads
- Installation files
- Cache files

Many Linux systems automatically clear `/tmp` when the system reboots.

18. `/usr`

Stores most user applications, libraries, manuals, and shared resources.

Common subdirectories:

`/usr/bin`

User commands

`/usr/lib`

Libraries

`/usr/share`

Documentation and shared files

`/usr/local`

Locally installed software

Think of `/usr` as the main software library where most installed applications live.

19. `/var`

Stores **variable data**, which changes frequently while the system is running.

Examples:

`/var/log`

System log files

`/var/mail`

User mail

`/var/spool`

Print jobs and queued tasks

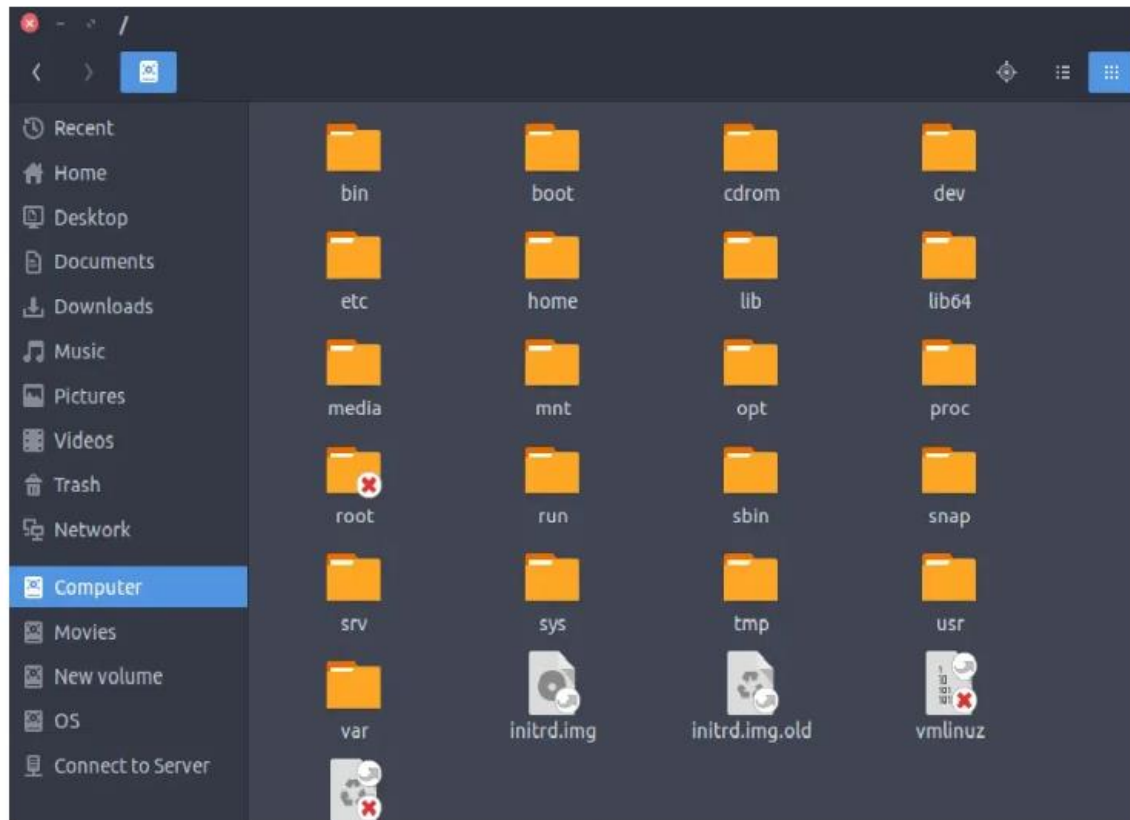
`/var/cache`

Application cache

1. / (Root)

At the top of every Linux file system is the root directory represented by a forward slash /. It's the base point, and no directory exists above it.

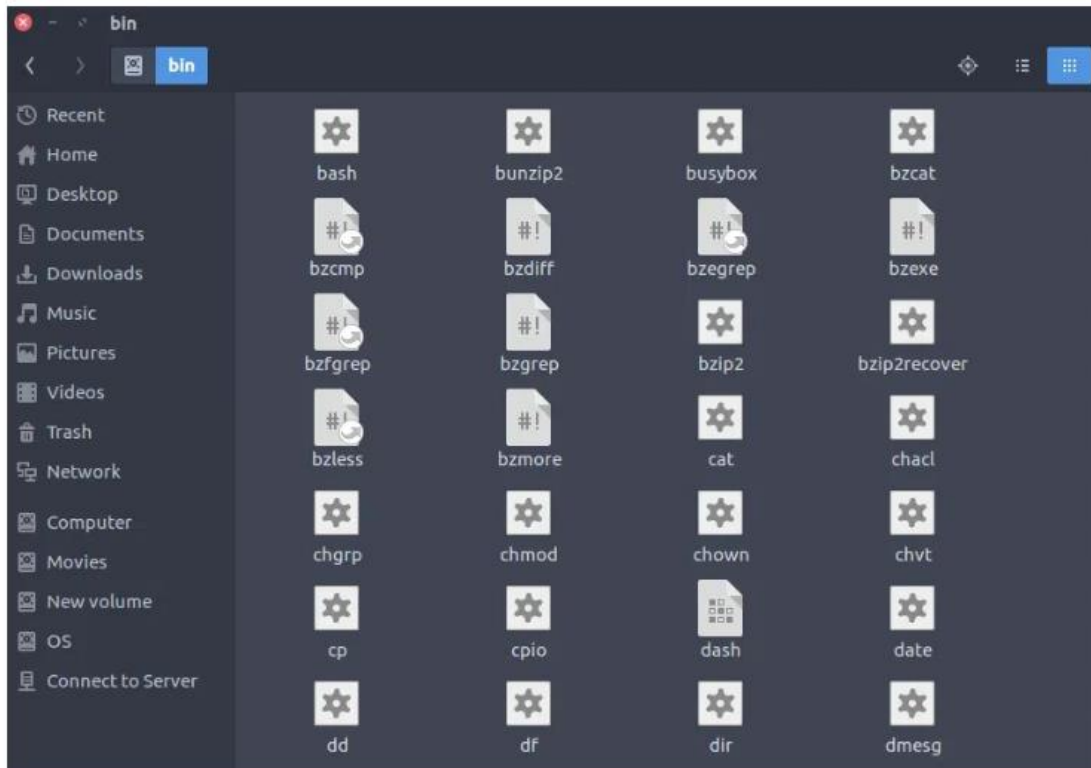
- If you look at the file system graphically, you'll see all other directories branching from this root directory
- Only the root user has permission to modify contents inside this directory.
- Regular users cannot make changes here. For example, if you attempt to create a file in / as a non-root user, you'll encounter a "Permission Denied" error



2. /bin

The /bin directory contains essential commands and binaries needed by all users, including `cp`, `ls`, [`ssh`](#), and [`kill`](#). These commands are universally available across user types.

- Contains binary executables
- Common linux commands you need to use in single-user modes are located under this directory
- Commands used by all the users of the system are located here e.g. `ps`, `ls`, `ping`, `grep`, `cp`



bin-linux-directory

Common Commands Found in /bin

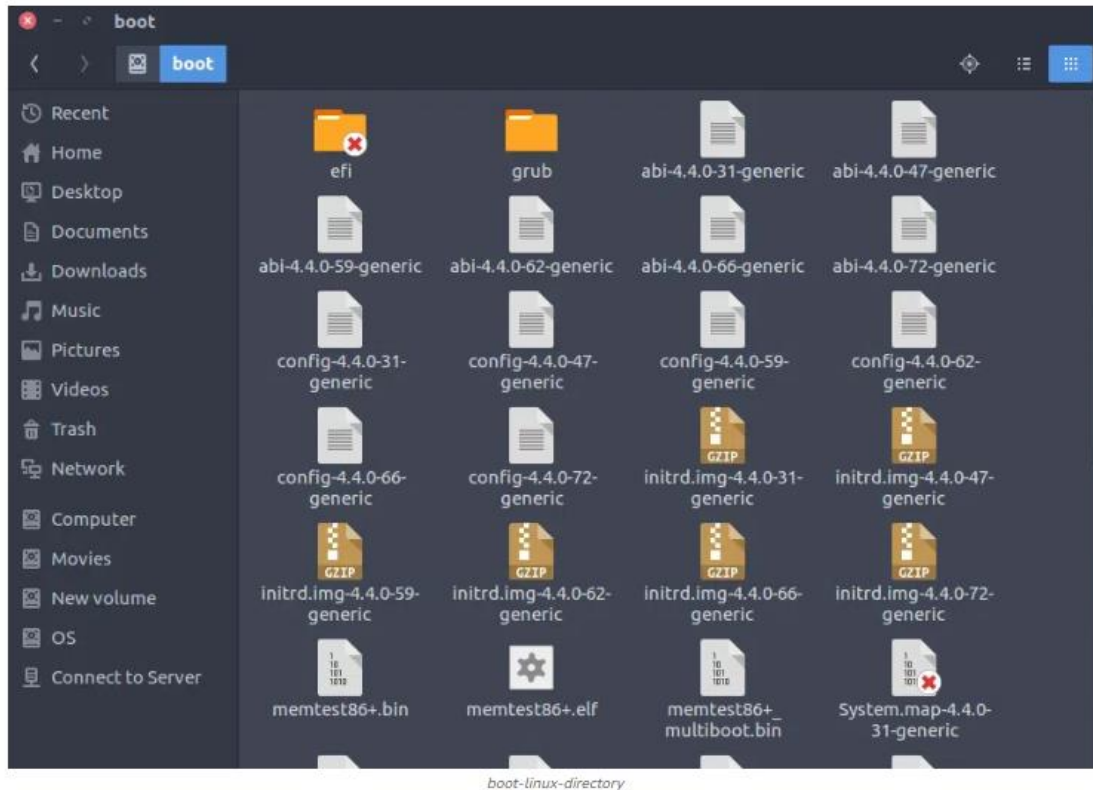
Examples include:

- `ls` → List files and directories.
- `cp` → Copy files.
- `mv` → Move or rename files.
- `cat` → Display file contents.
- `pwd` → Show the current directory.
- `mkdir` → Create a directory.
- `rm` → Remove files.

3./boot

The `/boot` directory contains the **files required to start (boot) the Linux operating system**.

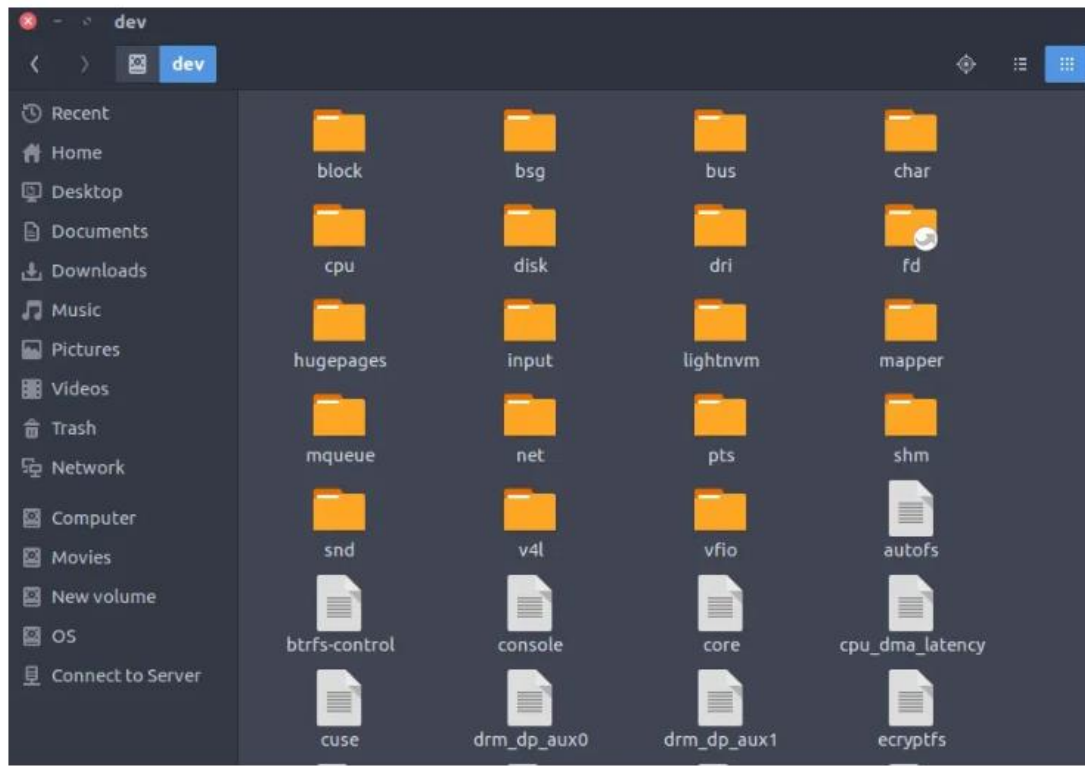
Without the files in `/boot`, Linux cannot boot properly.



4. /dev

Device files in Linux are stored in the /dev directory. These are special files that act as interfaces between hardware and software. Device files are of two types: block devices (e.g., hard drives) and character devices (e.g., microphones and speakers). Examples include /dev/sda1 for disk partitions.

- These include terminal devices, USB, or any device attached to the system
- **Example:** /dev/tty1, /dev/usbmon0

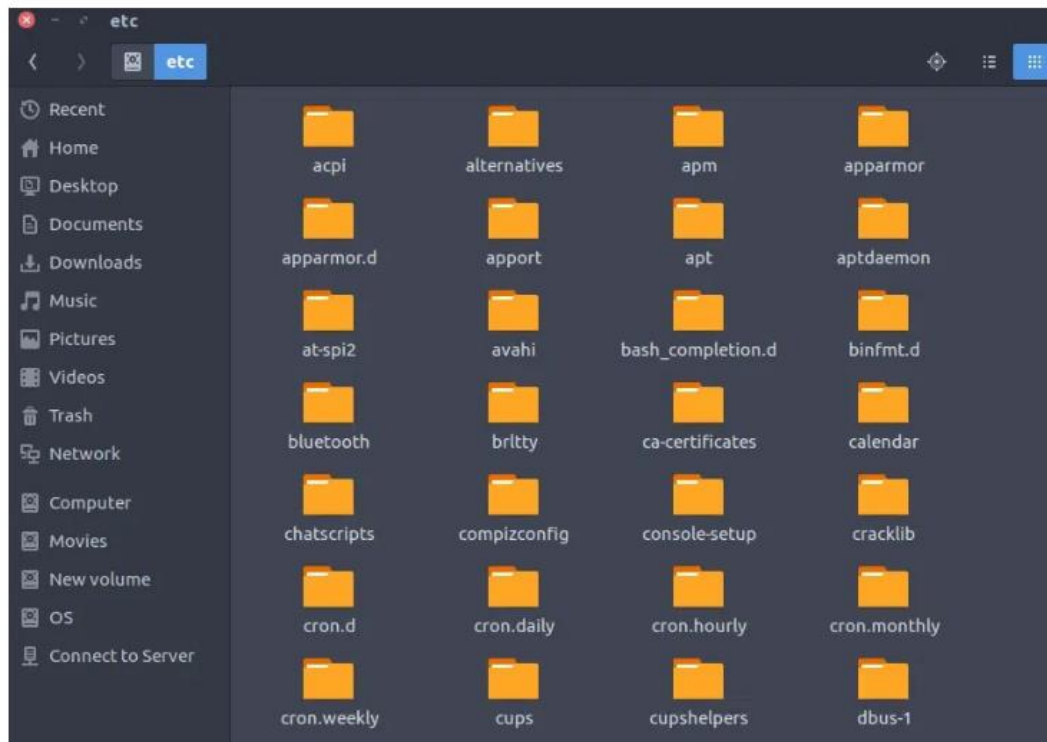


dev-linux-directory

5. /etc

Short for "Editable Text Configuration," /etc contains configuration files for system applications, users, services, and tools or it contains the Host-specific system-wide configuration files. For example, user details like UID and local addresses are defined here.

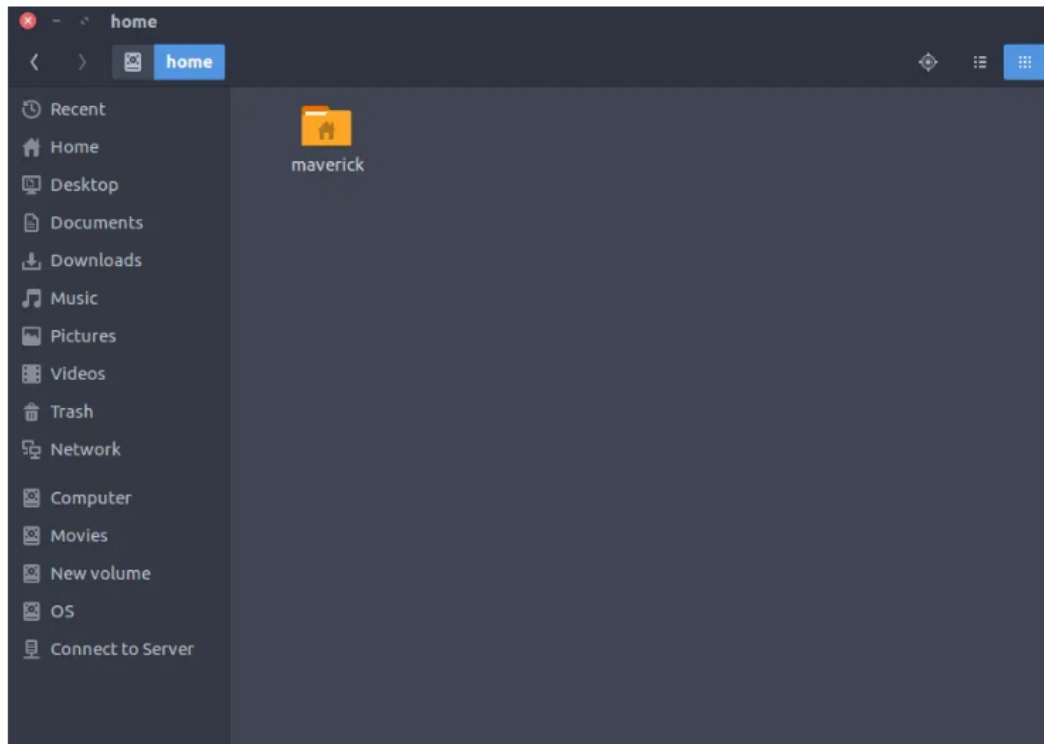
- This also contains startup and shutdown shell scripts used to start/stop individual programs
- **Example:** /etc/resolv.conf, /etc/logrotate.conf



6. /home

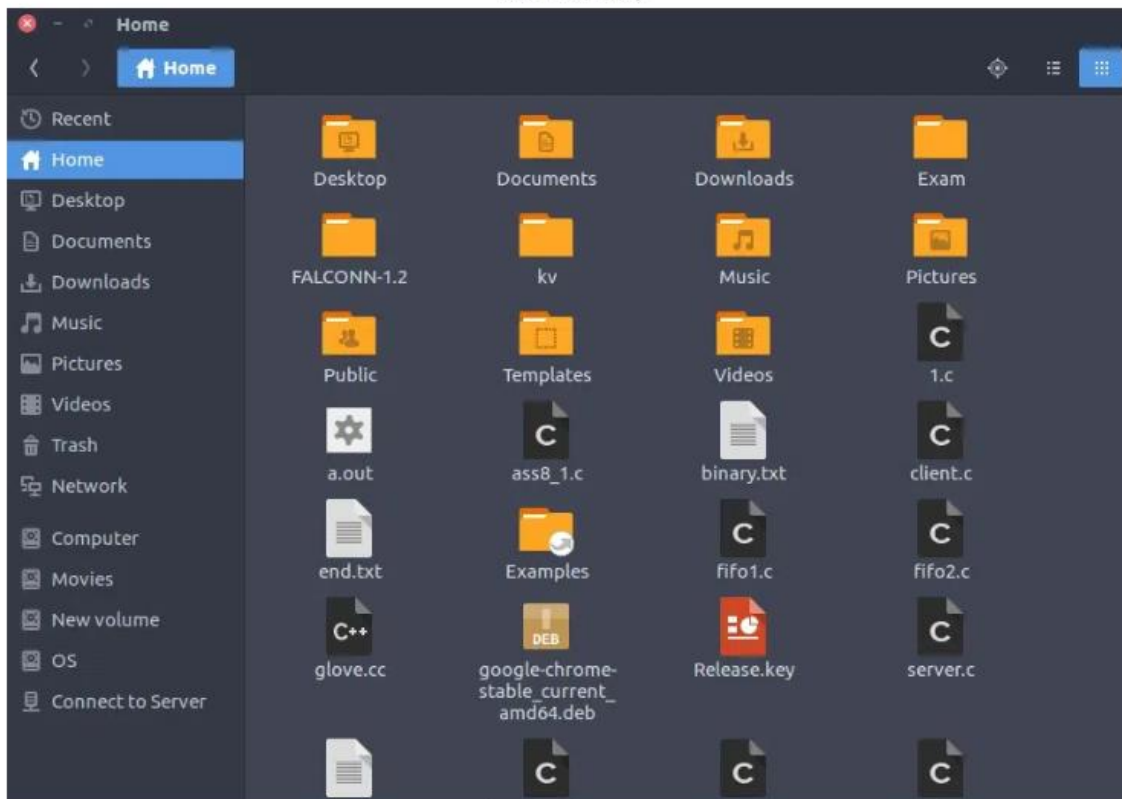
Every non-root user has a personal directory inside /home. For example, if your username is anshu, your personal directory would be /home/anshu. Each user can create, delete, or modify files only in their own home directory and access of another user's home depends on security configuration.

- Home directories for all users to store their personal files, containing saved files, personal settings, etc
- **Example:** /home/kishlay, /home/kv



home-home-directory

home-linux-directory

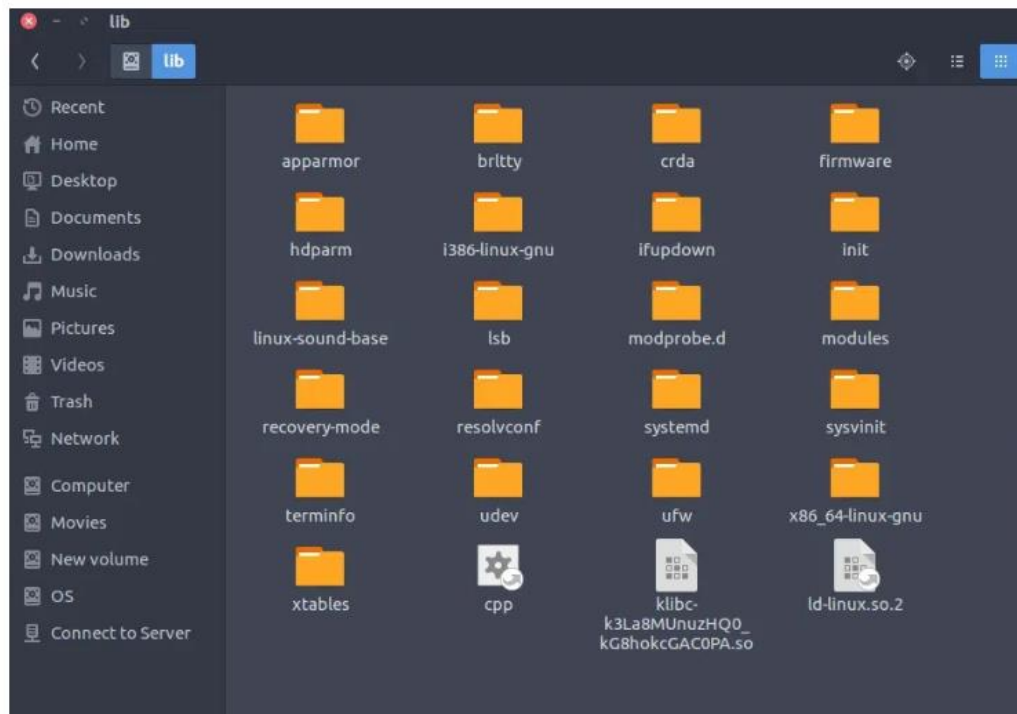


home-linux-directory

7. /lib

Applications require shared libraries to run, which are stored in /lib. These include dynamic libraries needed during runtime. For example, Apache server libraries are available here.

- Library filenames are either ld* or lib*.so.*
- **Example:** ld-2.11.1.so, libncurses.so.5.7

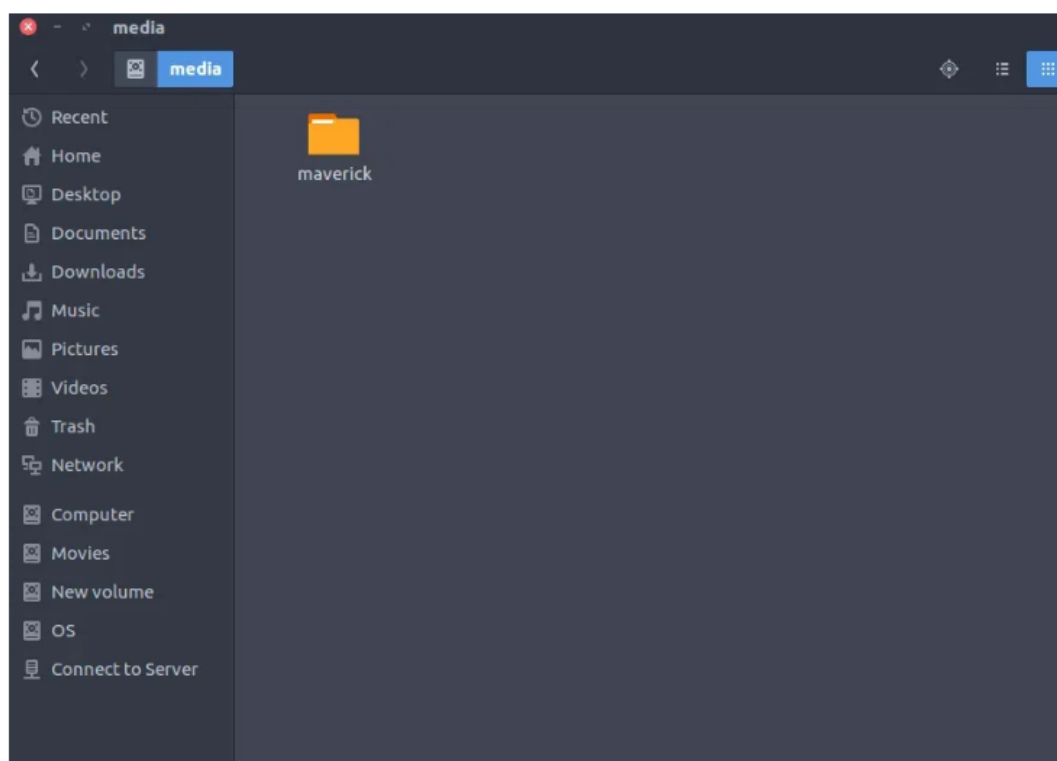


lib linux directory

8. /media

Devices like USBs, CDs, and pen drives are mounted under /media. For example, when a CD-ROM is inserted (appeared in FHS-2.3), its details will appear here.

- Temporary mount directory for removable devices
- **Examples:** /media/cdrom for CD-ROM; /media/floppy for floppy drives; /media/cdrecorder for CD writer

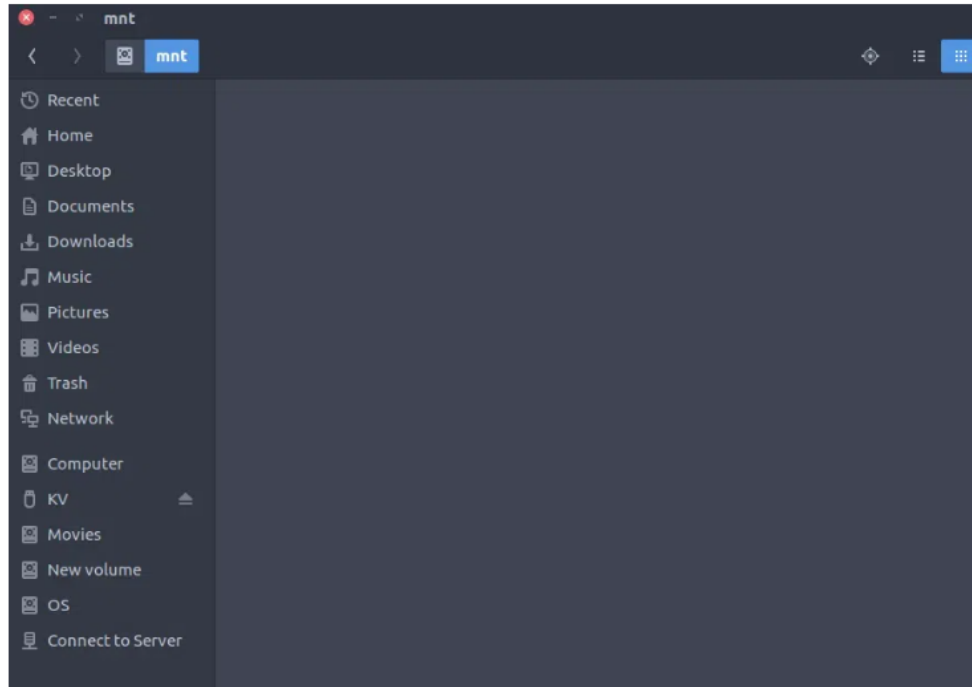


media-linux-directory

9. /mnt

When external drives are connected, they are temporarily mounted in /mnt. This is where their contents become accessible to the system.

- Temporary mount directory where sysadmins can mount filesystems

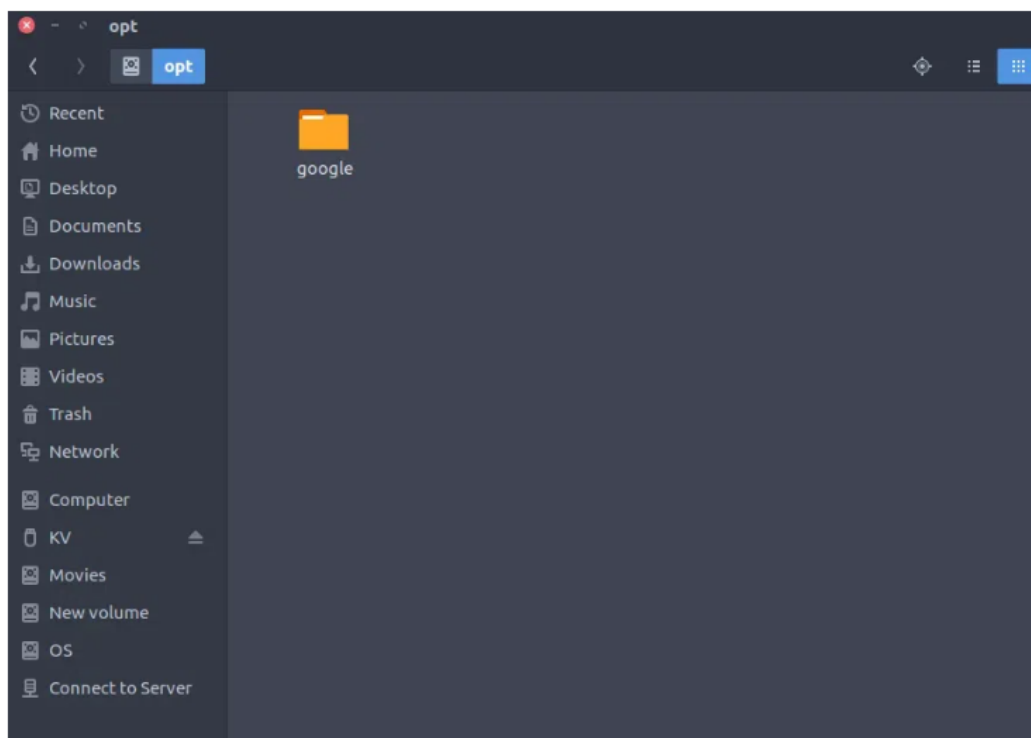


mnt-linux-directory

10. /opt

Third-party software and packages not part of the default system installation are stored in /opt. It includes their configuration and data files.

- Contains add-on applications from individual vendors
- Add-on applications should be installed under either /opt/ or /opt/subdirectory

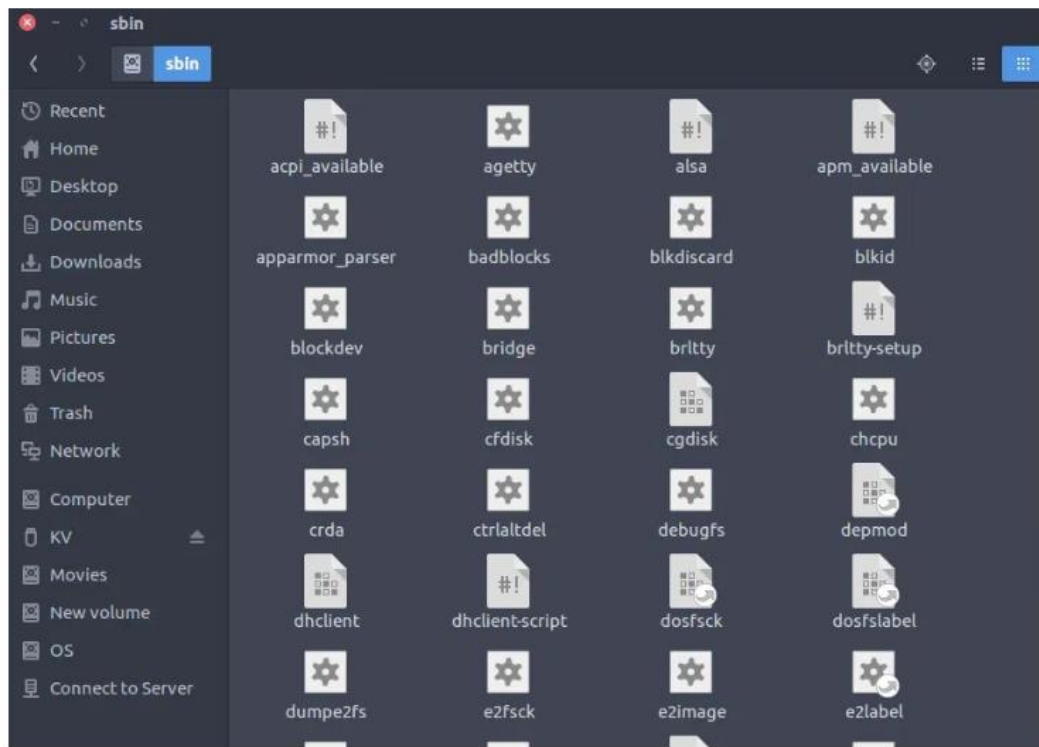


opt-linux-directory.

11. /sbin

This directory holds administrative binaries like iptables, firewall management tools, fsck, init, route etc. These binaries are primarily for system administrators and typically require root privileges to execute.

- Just like /bin, /sbin also contains binary executables.
- The linux commands located under this directory are used typically by system administrators, for system maintenance purposes
- **Example:** iptables, reboot, fdisk, ifconfig, swapon

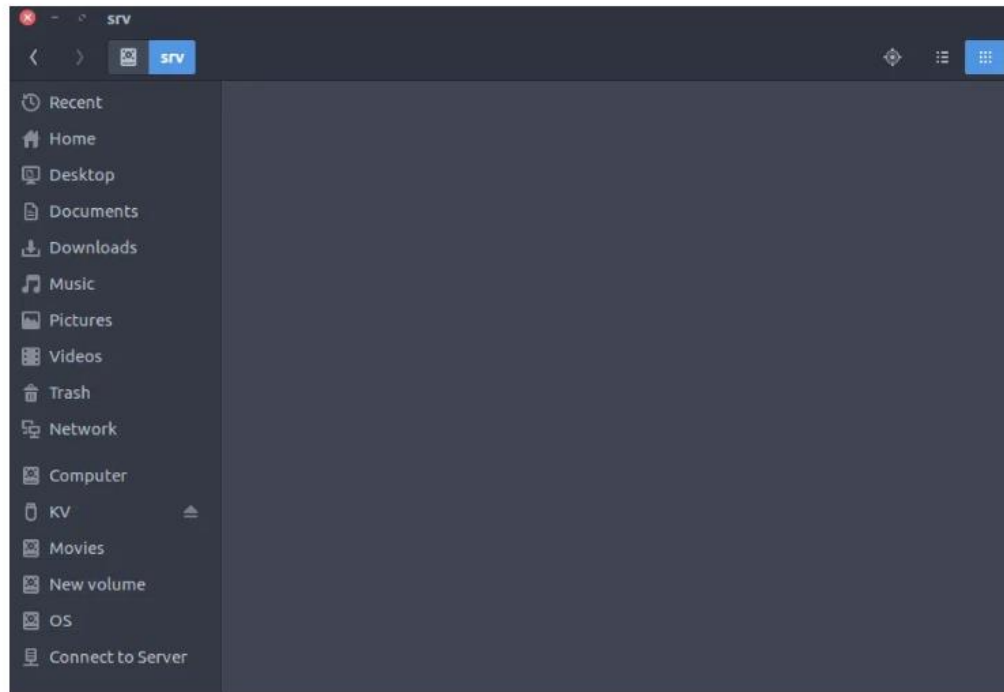


sbin-linux-directory

12. /srv

Site-specific data served by this system, such as data and scripts for web servers, data offered by FTP servers, and repositories for version control systems.

- `srv` stands for service
- Contains server specific services related data
- **Example:** `/srv/cvs` contains CVS related data

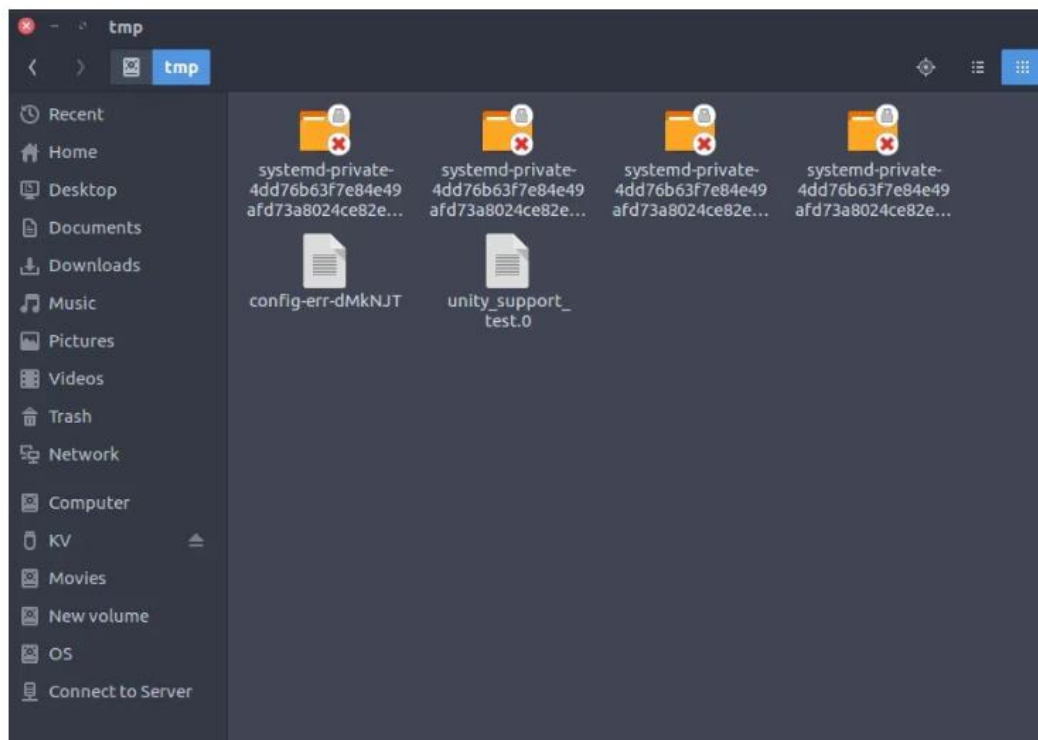


srv-linux-directory

13. /tmp

Programs create temporary files during execution, and these are stored in `/tmp`. These files are deleted automatically after the program finishes or when the system is restarted.

- Directory that contains temporary files created by system and users
- Files under this directory are deleted when the system is rebooted



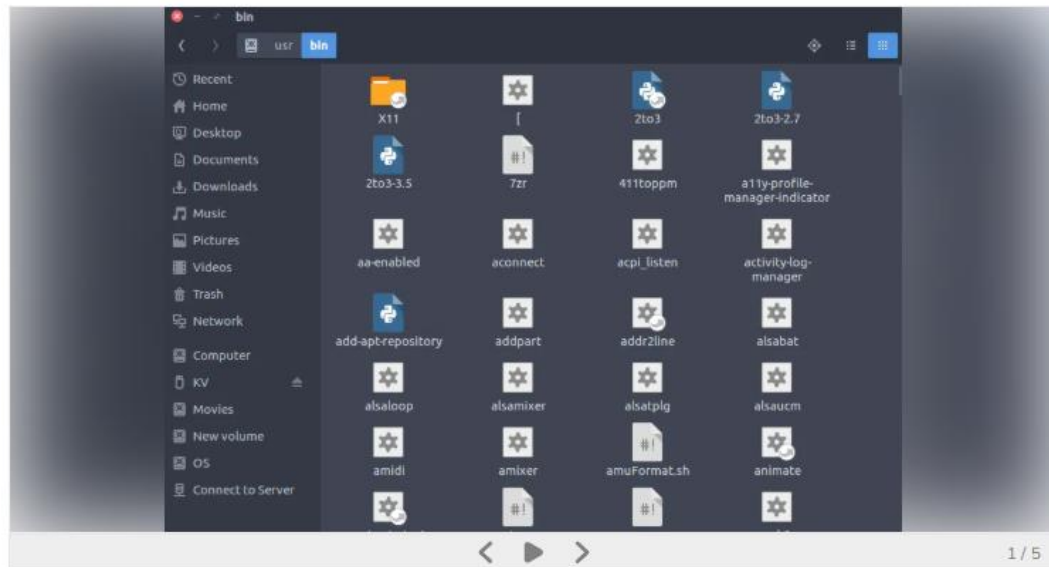
tmp-linux-directory

14./usr

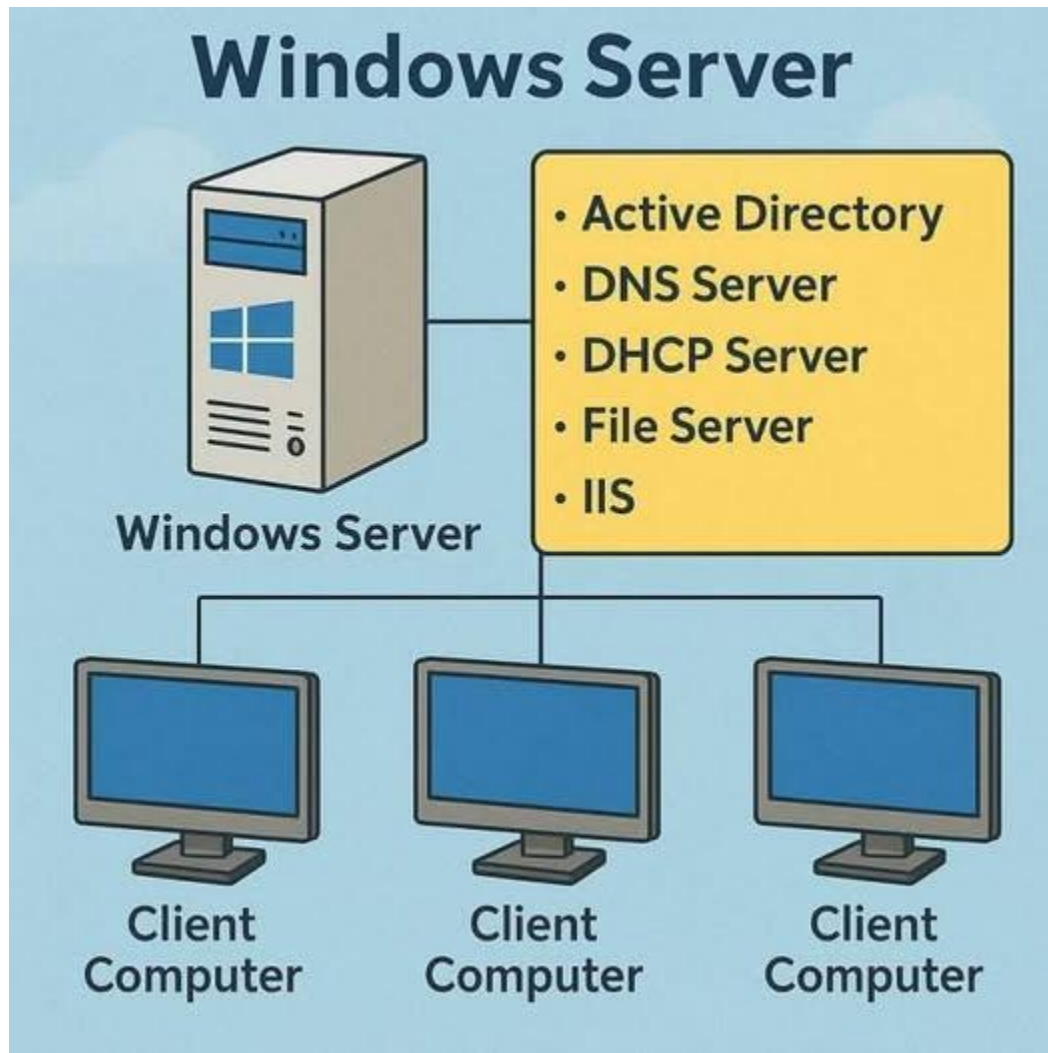
`/usr` stores **most of the system's programs and libraries**.

Examples:

- `/usr/bin` → Commands like `ls`, `cp`, `python`
- `/usr/lib` → Program libraries
- `/usr/share` → Documentation, icons, and other shared files
- `/usr/local` → Programs installed manually by the user/admin



Windows Server Operating System



Windows Server is an operating system that enables a computer (server) to provide services such as file sharing, web hosting, database management, user authentication, virtualization, and network administration to other computers (clients) on a network.

Features of Windows Server

1. Multi-user Support

- Supports many users simultaneously.
- Manages user accounts and permissions.

2. Active Directory (AD)

- Centralized management of users, computers, and groups.
- Provides authentication and authorization.
- 3. File and Print Services**
 - Shares files and printers across the network.
 - Controls access using permissions.
- 4. Web Server (IIS)**
 - Hosts websites and web applications.
 - Supports ASP.NET, PHP, HTML, and more.
- 5. DNS and DHCP**
 - **DNS (Domain Name System):** Converts domain names into IP addresses.
 - **DHCP (Dynamic Host Configuration Protocol):** Automatically assigns IP addresses.
- 6. Virtualization (Hyper-V)**
 - Runs multiple virtual machines on a single physical server.
 - Saves hardware costs.
- 7. Remote Desktop Services**
 - Allows users to access the server remotely.
 - Supports remote administration.
- 8. Storage Management**
 - RAID support
 - Storage Spaces
 - Disk Management
 - Data deduplication
- 9. Security**
 - Windows Defender
 - BitLocker Drive Encryption
 - Windows Firewall
 - Group Policy
 - Secure Boot
- 10. PowerShell**
 - Powerful command-line tool for automation and server management.

Active Directory (AD)

Active Directory (AD) is a directory service developed by Microsoft that stores and manages information about users, computers, groups, printers, and other network resources in a centralized database.

Roles and Functions of Active Directory

1. Centralized User Management

- Stores all user accounts in one location.
- Administrators can create, modify , disable, or delete user accounts centrally.

Example: A company administrator creates employee accounts from one server instead of configuring each computer individually.

2. Authentication

- Verifies users' usernames and passwords.
- Ensures that only authorized users can log in.

3. Authorization

- Determines what resources a user can access after logging in.
- Assigns permissions based on user roles or group membership.

4. Resource Management

Active Directory manages:

- Users
- Computers
- Printers
- Shared folders
- Applications
- Network devices

5. Group Policy Management

Administrators can enforce security and configuration settings through **Group Policy Objects (GPOs)**.

Examples:

- Disable Control Panel
- Set password policies
- Install software automatically
- Restrict USB devices

6. Organizational Structure

Resources are organized into:

- Domains (A university domain is **college.edu**.)
- Organizational Units (OUs) (**Teachers** group can access staff files, while the **Students** group cannot.)
- Groups

This makes administration easier.

7. Single Sign-On (SSO)

Users log in once and can access multiple authorized services without entering their credentials repeatedly.

8. Security Management

Active Directory:

- Enforces password policies
- Controls user permissions
- Audits user activities
- Helps protect network resources

Domain Controller (DC)

A **Domain Controller (DC)** is a Windows Server that hosts Active Directory and provides authentication, authorization, and centralized management services for the domain.

Roles and Functions of a Domain Controller

1. Hosts Active Directory Database

The Domain Controller stores the Active Directory database that contains:

- Users
- Groups
- Computers
- Policies
- Security information

2. User Authentication

The DC checks usernames and passwords whenever a user logs in.

Example: If the credentials are correct, the user is allowed to access the network.

3. Authorization

After successful authentication, the DC determines:

- Which files can be accessed
- Which printers can be used
- Which applications are allowed

4. Policy Enforcement

The Domain Controller applies Group Policies to users and computers automatically.

5. Replication

If multiple Domain Controllers exist, they synchronize Active Directory data so that all servers have the latest information.

Benefits:

- High availability
- Fault tolerance
- Backup of directory information

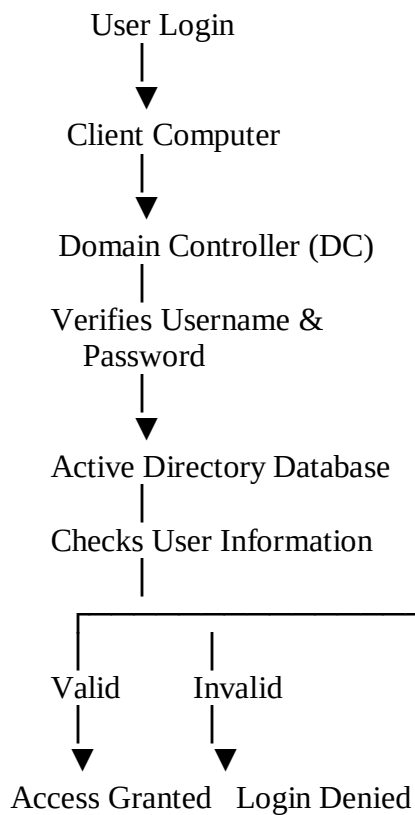
6. DNS Integration

A Domain Controller often works with **DNS (Domain Name System)** to help clients locate domain resources and services.

7. Centralized Administration

Administrators manage the entire network from one place instead of configuring each computer individually.

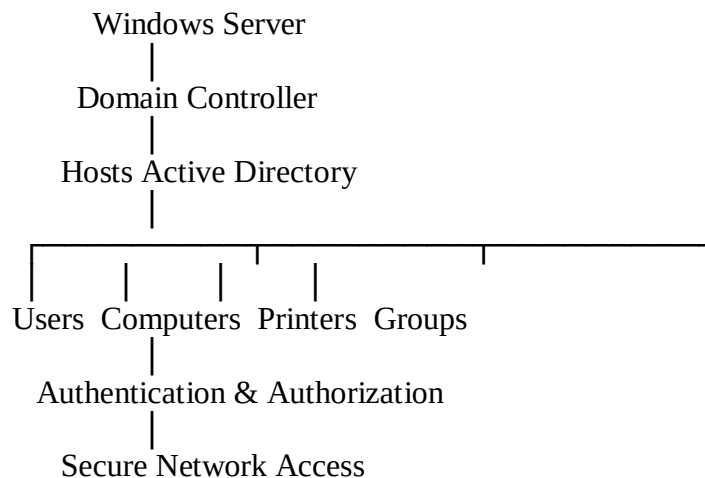
Working Process



Difference Between Active Directory and Domain Controller

Feature	Active Directory (AD)	Domain Controller (DC)
Definition	Directory service	Server that hosts AD
Type	Software/service	Physical or virtual server
Purpose	Stores and organizes directory information	Authenticates users and manages the domain
User Accounts	Stores user accounts	Uses AD to verify user accounts
Authentication	Provides identity information	Performs the authentication process
Resource Management	Organizes network resources	Controls access to resources
Multiple Instances	One AD domain can be served by multiple DCs	Organizations often deploy multiple DCs for redundancy

Relationship Between AD and DC



Advantages

Active Directory

- Centralized administration
- Improved security
- Easy resource management
- Single Sign-On (SSO)
- Simplifies user and group management

Domain Controller

- Secure authentication
 - Centralized login management
 - Policy enforcement
 - Data replication for reliability
 - Better network security
-

Real-Life Example

Suppose a university has **500 computers**.

1. A student enters a username and password on any lab computer.
2. The computer sends the login request to the **Domain Controller**.
3. The Domain Controller checks the credentials stored in **Active Directory**.
4. If the credentials are valid, the student is allowed to log in.
5. Group Policies are applied automatically, such as restricting software installation or access to specific folders.

Windows Server Roles and Features

In **Windows Server**, **Roles** and **Features** are installed to enable the server to perform specific tasks and provide additional functionality.

- **Server Role:** Defines the **main job** of the server.
- **Server Feature:** Adds **extra capabilities** to support the server or its roles.

Windows Server Roles

A **Server Role** is the primary function that a Windows Server performs for clients on a network. A server can have one or more roles installed depending on the organization's needs.

Common Windows Server Roles

Role	Description
Active Directory Domain Services (AD DS)	Provides centralized management of users, computers, groups, and domains. Enables the server to become a Domain Controller.
DNS Server	Resolves domain names (e.g., <code>www.example.com</code>) into IP addresses.
DHCP Server	Automatically assigns IP addresses and network settings to client computers.
File and Storage Services	Stores, manages, and shares files and folders over the network.
Print and Document Services	Shares and manages network printers.
Web Server (IIS)	Hosts websites, web applications, and web services.
Hyper-V	Creates and manages virtual machines (VMs).
Remote Desktop Services (RDS)	Allows users to remotely access desktops and applications.
Windows Server Update Services (WSUS)	Distributes Microsoft updates to computers on the network.
Network Policy and Access Services (NPAS)	Controls network authentication and access policies.

Functions of Server Roles

- Centralized user and computer management
- File and printer sharing
- Website and application hosting
- Automatic IP address allocation
- Virtualization support
- Remote access services
- Network security and authentication

Windows Server Features

A **Feature** is an optional software component that provides additional functionality to Windows Server. Features usually support server roles or enhance system management and security.

Common Windows Server Features

Feature	Description
.NET Framework	Provides libraries required to run many Windows applications.
Windows PowerShell	Automates administration through scripting and command-line tools.
BitLocker Drive Encryption	Encrypts disks to protect data from unauthorized access.
Windows Server Backup	Creates backups and restores server data.
Failover Clustering	Provides high availability by allowing another server to take over if one fails.
Network Load Balancing (NLB)	Distributes client traffic across multiple servers.
Multipath I/O (MPIO)	Improves storage performance and fault tolerance.
Remote Server Administration Tools (RSAT)	Enables remote management of Windows Servers.
Telnet Client	Provides command-line access to remote systems using the Telnet protocol.
SNMP Service	Monitors and manages network devices using SNMP.

Difference Between Roles and Features

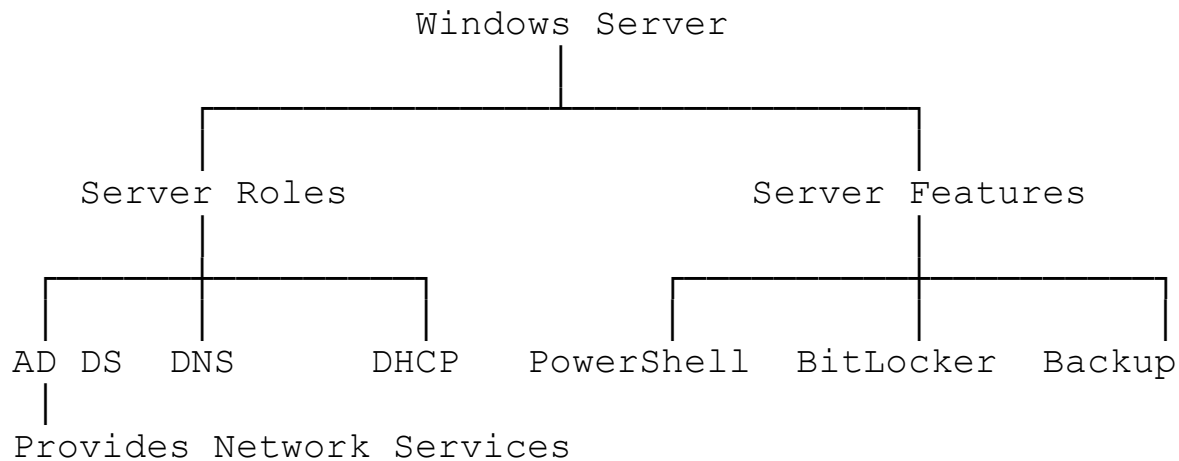
Server Roles	Server Features
Define the server's main purpose	Add optional capabilities
Provide core network services	Support or enhance roles
Installed according to the server's function	Installed as needed

Server Roles

Examples: AD DS, DNS, DHCP, IIS

Server Features

Examples: PowerShell, BitLocker, .NET Framework



Example

A school installs one Windows Server.

Installed Roles

- **AD DS** → Manages student and teacher accounts.
- **DNS** → Resolves domain names.
- **DHCP** → Assigns IP addresses automatically.
- **File Server** → Stores students' documents.
- **Print Server** → Shares printers.

Installed Features

- **BitLocker** → Encrypts the server's storage.
- **PowerShell** → Automates administrative tasks.
- **Windows Server Backup** → Backs up important data.
- **Failover Clustering** → Ensures services remain available during server failures.

Advantages

- Centralized administration
- Improved security
- Easy resource sharing
- High reliability and availability

- Better performance
- Scalability for growing organizations

Windows Server Security and Group Policies

Windows Server Security refers to the measures and tools used to protect servers, users, data, and network resources from unauthorized access, malware, and cyber threats.

Group Policy is a Windows Server feature that allows administrators to centrally configure and enforce security settings and system configurations for users and computers in an **Active Directory** domain.

Windows Server Security

Windows Server Security is the collection of security features, policies, and technologies that protect the server and network resources by ensuring only authorized users can access them.

Objectives of Windows Server Security

- Protect users and computers from unauthorized access.
- Secure files, folders, and network resources.
- Prevent malware and cyberattacks.
- Enforce security policies.
- Ensure data confidentiality, integrity, and availability (CIA).

Major Security Features

1. Active Directory (AD)

- Authenticates users and computers.
- Centralizes user and resource management.
- Controls access to network resources.

2. Windows Defender

- Protects against viruses, malware, spyware, and ransomware.
- Provides real-time protection.

3. Windows Firewall

- Monitors and filters incoming and outgoing network traffic.
- Blocks unauthorized network connections.

4. BitLocker Drive Encryption

- Encrypts hard drives.
- Protects sensitive data if the server is lost or stolen.

5. NTFS Permissions

Controls access to files and folders.

Examples:

- Read
- Write
- Modify
- Full Control

6. User Authentication

Methods include:

- Username and password
- Smart cards
- Multi-Factor Authentication (MFA) (if configured)

7. Auditing

- Records login attempts.
- Monitors file access.
- Helps detect suspicious activities.

8. Windows Server Backup

- Protects data through regular backups.
- Supports disaster recovery.

Group Policy

Group Policy is a Windows Server feature that enables administrators to centrally manage security settings, user environments, and computer configurations across an Active Directory domain.

Instead of configuring each computer individually, Group Policy allows one policy to be applied to many users and computers.

Functions of Group Policy

1. Password Policy

- Enforces password complexity.
- Sets minimum password length.
- Defines password expiration.
- Prevents password reuse.

2. Account Lockout Policy

- Locks a user account after several failed login attempts.
- Protects against password-guessing attacks.

3. Software Installation

- Installs approved software automatically.
- Prevents installation of unauthorized programs.

4. Desktop Restrictions

Administrators can:

- Hide Control Panel.
- Disable Task Manager.
- Prevent changes to desktop settings.
- Restrict access to Command Prompt.

5. Windows Update Policy

- Controls how and when updates are installed.
- Ensures all computers receive security updates.

6. Security Settings

- Configure Windows Firewall.
- Enable auditing.
- Set user rights and permissions.
- Control security options.

7. Folder Redirection

Stores user folders (such as Documents and Desktop) on a central server for easier backup and management.

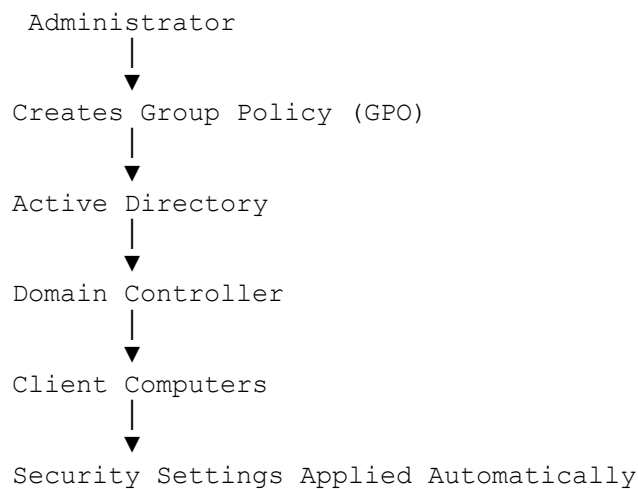
8. USB and Device Restrictions

- Disable USB storage devices.
- Block unauthorized hardware.
- Prevent data theft.

Types of Group Policies

Type	Purpose
Local Group Policy	Applies to one computer only.
Domain Group Policy	Applies to all users and computers in an Active Directory domain.

Working Process



Example

A school has **300 computers**.

The administrator creates a Group Policy that:

- Requires passwords to be at least 10 characters.
- Disables USB storage devices.
- Automatically installs antivirus software.
- Hides the Control Panel for students.
- Enables Windows Firewall.

When students log in, these settings are applied automatically to every computer in the domain.

Difference Between Windows Server Security and Group Policy

Windows Server Security

Protects the server and network from threats

Includes Firewall, Defender, BitLocker, NTFS permissions, and auditing

Focuses on overall system protection

Uses multiple security technologies

Group Policy

Centrally configures and enforces settings

Includes password policies, software deployment, desktop restrictions, and update management

Focuses on centralized configuration and management

Uses **Group Policy Objects (GPOs)** to apply settings

Advantages

Windows Server Security

- Protects data and network resources.
- Prevents unauthorized access.
- Detects and blocks malware.
- Improves overall network security.
- Supports compliance through auditing.

Group Policy

- Centralized administration.
- Consistent security settings.
- Saves time by configuring many computers at once.
- Reduces configuration errors.
- Improves security by enforcing organizational policies.

Linux File Permission and Security

Linux file permissions are a security mechanism that controls **who can read, write, or execute files and directories**. Every file and directory has permissions that protect it from unauthorized access.

Purpose:

- Protect sensitive data
- Prevent unauthorized modification
- Restrict execution of malicious programs
- Maintain system security

2. Linux Permission Classes

Linux divides users into **three categories**:

User Class Symbol		Description
Owner	u	Person who owns the file
Group	g	Users belonging to the file's group
Others	o	Everyone else

Example:

```
-rwxr-xr--
```

This permission is divided as:

Owner	Group	Others
rwx	r-x	r--

3. Types of Permissions

Linux has **three basic permissions**.

Permission Symbol Value			Meaning
Read	r	4	View file contents
Write	w	2	Modify file
Execute	x	1	Run program/script

Read Permission (r)

Allows user to:

- Open file
- View contents
- Copy file

Example

```
cat notes.txt
```

Without read permission:

```
Permission denied
```

Write Permission (w)

Allows user to

- Edit file

- Rename file
- Delete file (if directory permits)

Example

```
nano notes.txt
```

Without write permission

```
Read-only file
```

Execute Permission (x)

Allows

- Execute program
- Run shell script

Example

```
./program
```

Without execute permission

```
Permission denied
```

4. Permission Representation

Example

```
-rwxr-xr--
```

Breakdown

Position	Meaning
-	File type
rwx	Owner permissions
r-x	Group permissions
r--	Others permissions

File Types

Symbol	Meaning
-	Regular file
d	Directory
l	Symbolic link
c	Character device

Symbol	Meaning
--------	---------

b	Block device
---	--------------

Example

```
drwxr-xr-x
```

"d" means directory.

5. Numeric (Octal) Permissions

Each permission has a numeric value.

```
Read = 4
Write = 2
Execute = 1
```

Add them together.

Permission Value

rwX	7
-----	---

rw-	6
-----	---

r-x	5
-----	---

r--	4
-----	---

-wX	3
-----	---

-w-	2
-----	---

--x	1
-----	---

---	0
-----	---

Example

```
755
```

Means

```
Owner   = 7 = rwX
Group   = 5 = r-x
Others  = 5 = r-x
```

Another example

```
644
```

Means

```
Owner = rw-
Group = r--
Others = r--
```

6. Viewing Permissions

Use

```
ls -l
```

Example

```
-rwxr-xr-- 1 user staff 2048 Jul 10 report.sh
```

Explanation

-rwxr-xr--	Permission
1	Number of links
user	Owner
staff	Group
2048	File size
Jul 10	Date
report.sh	File name

7. Changing Permissions

chmod Command

Used to modify permissions.

Symbolic Method

```
chmod u+x file.sh
```

Adds execute permission to owner.

Remove permission

```
chmod g-w file.txt
```

Add read permission

```
chmod o+r file.txt
```

Numeric Method

Example

```
chmod 755 script.sh
```

Means

```
Owner = rwx  
Group = r-x  
Others = r-x
```

Example

```
chmod 600 secret.txt
```


Only owner can read and write.

8. Changing Ownership

chown

Changes owner.

Example

```
sudo chown ram file.txt
```

Owner becomes "ram".

Change Group

```
sudo chown ram:students file.txt
```

Owner

ram

Group

students

chgrp Command

Changes only group.

```
sudo chgrp teachers notes.txt
```

9. Directory Permissions

Permissions work differently for directories.

Permission	Meaning
r	List files
w	Create/Delete files
x	Enter directory

Example

```
drwxr-xr-x
```

Owner can

- Enter

- Create
- Delete
- List files

Others

- Can enter
- Can list
- Cannot modify

10. Default Permissions (umask)

When creating files:

Default permissions are controlled by **umask**.

Check

`umask`

Example

`0022`

Default permissions

Files

`644`

Directories

`755`

11. Special Permissions

(A) SUID (Set User ID)

When executable runs, it executes with the **owner's privileges**.

Symbol

`s`

Example

`-rwsr-xr-x`

Numeric

`4000`

Example

```
chmod 4755 file
```

Common use

```
passwd
```

Allows normal users to change passwords.

(B) SGID (Set Group ID)

Program runs using **group privileges**.

Numeric

```
2000
```

Directory example

Files created inherit the directory's group.

(C) Sticky Bit

Used on directories.

Only file owner can delete their own files.

Numeric

```
1000
```

Example

```
drwxrwxrwt
```

Common example

```
/tmp
```

12. Access Control Lists (ACL)

ACL provides **more flexible permissions** than the standard owner/group/others model by allowing permissions for specific users or groups.

Example:

```
setfacl -m u:alice:rwX project.txt
```

View ACL:

```
getfacl project.txt
```

13. Linux File Security Best Practices

- Follow the **Principle of Least Privilege** (give only required permissions).
- Avoid using `777` permissions unless absolutely necessary.
- Use strong passwords and secure user accounts.
- Regularly audit permissions using `ls -l` and `find`.
- Restrict root access and use `sudo` instead of logging in as root.
- Keep the system updated with security patches.
- Back up important files regularly.